

DATA 266 - Homework 4 Findings

Building a Mini GPT from Scratch (Multi-Head, Character Level)

Personal Parameters

Parameter	Value
SID4	9486
SEED	9486
SLICE	486
HP_ID	0
CLS_A	6
CLS_B	0

I used SEED 9486 for Python, NumPy, and PyTorch reproducibility throughout the notebook, for weight initialization, DataLoader shuffling, and every sampling call. The rest of the personal parameters are reported for consistency with the standing Assignment 1 instructions, since this homework does not use SLICE, HP_ID, CLS_A, or CLS_B to pick a dataset variant or hyperparameter arm.

Introduction

For this assignment I built a decoder only, GPT style Transformer entirely out of plain PyTorch layers: `nn.Linear`, `nn.LayerNorm`, `nn.GELU`, and `nn.Embedding`. I did not use `nn.Transformer`, `nn.MultiheadAttention`, or any HuggingFace transformer classes anywhere in the code. Starting from the single head demo model (Demo_4_Mini_GPT.ipynb), I extended it to four head masked self attention, a sequence length of 128, and four decoder blocks, trained it on the given shakespeare.txt excerpt, and compared greedy, temperature, and top-k decoding. Every result in this report comes from the executed HW4_Mini_GPT.ipynb notebook, run on Google Colab with a GPU.

What's in this submission

Part	What was completed
Part 1	Character level tokenization of shakespeare.txt, a sliding window dataset with a sequence length of 128, and two deco
Part 2	Multi-head (4 head) masked self-attention built by hand, a Transformer decoder block (attention, residual connection, L
Part 3	Training with the Adam optimizer for 6 epochs, a training loss versus epoch plot, and a saved model checkpoint.
Part 4	Greedy decoding, temperature sampling at 0.5, 1.0, and 1.5, and top-k sampling at k=5 and k=20, all starting from the
Part 5	A comparison of the three decoding methods based on the actual generated text.
Verification	Attention row sums, zero attention weight on future positions, model output shape, and the hidden dimension dividing e

Part 1: Data Preparation

Dataset and tokenization

shakespeare.txt is the Week 4 dataset, the opening of the balcony scene from Romeo and Juliet, and I loaded it straight from disk with no changes and no substitute text. Tokenizing it at the character level gave a vocabulary of 49 distinct characters out of 1,738 total characters in the file.

Setting	Value
Total characters	1,738
Vocabulary size	49
Sequence length	128
Sliding window training examples	1,609 (length of text minus sequence length minus 1)
Input and target relationship	the target is the same 128 character window as the input, shifted forward by exactly one character

An example input and target pair, decoded back to text

Input: "ROME0:\nBut, soft! what light through yonder window breaks?\nIt is the east, and Juliet is the sun.
Target: "OME0:\nBut, soft! what light through yonder window breaks?\nIt is the east, and Juliet is the sun."

The target is exactly the input shifted forward by one character. I checked this directly in the notebook with a tensor equality comparison rather than just eyeballing it.

Model configuration

Setting	Value
Architecture	MiniGPT: token and positional embeddings, a stack of decoder blocks, a final LayerNorm, and a linear layer
Attention	manual multi-head masked self-attention, with separate Q, K, and V projections and a causal mask
Hidden dimension	128
Number of heads	4 (head dimension is 128 divided by 4, which is 32)
Number of decoder blocks	4
Feed forward network	a linear layer from 128 to 512, GELU, then a linear layer back down to 128
Total parameter count	822,321
Optimizer	Adam, learning rate 3e-4
Batch size	32
Epochs	6
Loss	cross entropy between the model's next character predictions and the shifted targets
Device	cuda, running on Google Colab's GPU
Reproducibility	SEED 9486 applied to Python, NumPy, and PyTorch, including the CUDA generator

Part 2: Multi-Head Masked Self-Attention

Given an input of shape batch by sequence length by hidden dimension, three separate linear layers compute the query, key, and value projections. Each one gets reshaped so the hidden dimension is split across four heads, giving four independent attention subspaces that run in parallel. Attention scores come from the query and key dot product, scaled down by the square root of the head dimension, and before the softmax step I apply a lower triangular boolean mask so that position i's score against any later position becomes negative infinity and ends up with zero probability. Afterward, the four heads' outputs are concatenated back together and passed through one more linear layer.

Checking the causal mask

Check	Result
Attention row sums (should all be about 1)	minimum 0.9999998212, maximum 1.0000001192
Total attention weight on future positions	0.0
Model output shape on a real training batch	32 by 128 by 49, matching batch size, sequence length, and vocabulary
Hidden dimension divisible by head count	128 divided by 4 is exactly 32

These are checked directly on the actual tensors in the notebook, not just by looking at them. Every attention row sums to about 1, like a proper softmax distribution should, and the total attention weight placed on future tokens comes out to exactly zero, which confirms the causal mask is doing its job.

Part 3: Training

I trained the model for 6 epochs with the Adam optimizer at a learning rate of $3e-4$ and a batch size of 32, using cross entropy loss between the model's predictions and the shifted targets from the sliding window dataset. The loss went down every single epoch:

Epoch	Average Loss
1	2.9655
2	2.3758
3	2.1977
4	2.0952
5	1.9809
6	1.8303

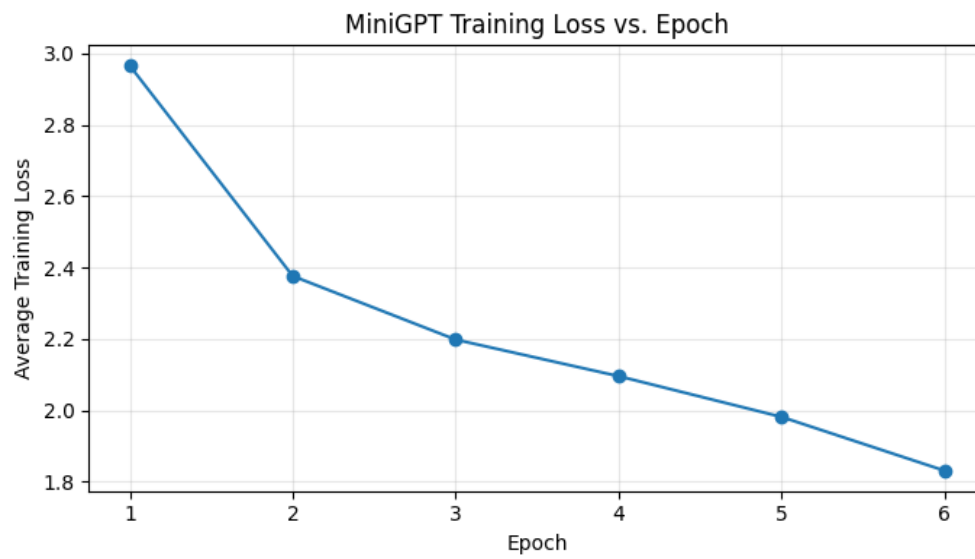


Figure 1. Training loss versus epoch, six epochs, batch size 32, run on Colab's GPU.

Part 4: Sampling, starting from "ROMEO:"

Each sample below generates 300 new characters from the same trained model and the same prompt. I reset the seed to 9486 right before every one of the six calls so the results are reproducible.

Greedy decoding

```
ROMEO:
Romeomeeo, theou nat thy nathamer that namer nd nar na nar whe
Or nor nor nathe
O nor nathame
Whathath tha nath name
Whathame n nor name
O nor na nor nor na na nor a name
Wha na a na na na a a name
OMor nor nor ame
OMor name
OMor name
OMor ame
OMEOMEO:
O:
O:
O:
O:
O ame, nor nor name, name, nor nor
```

Temperature sampling

Temperature 0.5

```
ROMEO:
MEO:
I that thar is is eeeat t nou eathel my spe,
Hengunondo metho butheno withes inr gheo il in bu hes my sheyespatherenor art werer irer t tis
Whe'llll t,

Wherer meereono ar mere ar athe ar mer areathe spu an ame sin ather n sher ther er alllllll baven ovenve,
Son al ben ame s:
By ber beall anven
```

Temperature 1.0

```
ROMEO:
My name:
So be
Aname hy , a w soll gullll m:

JULIEO:
Heam:
Soteo brt that t'l nd, r g:
I nono sow hapeeshy I nonam:
IEThe art ar ne isot t MiHewhalley t, thefouf meervew ir merer,
O ger 'll cr her ticar mor bucatheert?
Jpen llroit JULIETherat antharen or nve, n me bermou met sha me hee?

By came
```

Temperature 1.5

```
ROMEO;;
Henver nor beren.
Ar ge, cowveoll gu Rolomo.

Jveorfeou am:
```

SoteoOMrt.

JNol'AULRoar?g:

orefor st; bapeme bulyo! am:

Ifome art arellAn bu t MiHewhad my t, bu fovermeerveLw irblerm s, bulol'll crd,;

Ttice, mornerffosnelrthme.

LI'rrit Jrtetr, hat ant;

Ro! o? nve,tu n.

Sormf d It s, moo.

Ne?

ortcr d

Top-k sampling

k = 5

ROMEO:
 Whu nal ame amor ar whicee, wispat th theou thor ind illond,
 I't wo bl be I thar near I tororth be an ane aithyst art;
 Talr art ar n a t my merther ther t,
 Wherthar nguller t alert shat
 Anar t t me;
 Thicar, nonor athinorther, ther t a mar na hat anor the th nor th nor, thou me m:
 By the tha natherd

k = 20

ROMEO:
 My name:
 So be
 Aname hy nor w soll RO:
 Ilome
 I I meofeome
 I I ateo but that t'ld, I'r d:
 I nonit bu napeme nd:
 ROMEO:
 If, mee; car no beor t MEHewhalleant, but nafameer bew ir l r lld bulorer scr her tices;
 By befut nelrtheren llro tiswieor,
 'lll nthavet ot nve, n n.
 Sot n d I s:
 I me heereonace I

Comparing the decoding methods

With only 6 epochs of training on a text that's under 2,000 characters long, none of these six samples come out as real, fluent Shakespeare, but the differences between the decoding methods still show up clearly. Greedy decoding fell into a repeating loop almost right away, cycling through fragments like "nor nor nath" and "O nor nathame" over and over. That's a pretty classic failure mode for argmax on a model that hasn't been trained long enough: once it becomes confident that one of these fragments should follow, there's no mechanism to break out of the loop.

Temperature sampling showed a clear progression as I raised the temperature. At 0.5 the text still repeats letters somewhat but reads a little more naturally than greedy. At 1.0 it starts mixing in odd capitalization and punctuation. At 1.5 it's the most scrambled of all six samples, with unusual characters showing up in the middle of words. Top-k sampling followed a similar pattern: k=5 stayed close to greedy's repetition problem, while k=20 was noticeably more varied and felt a lot like the temperature 1.0 sample, which makes sense since both are drawing from a wider pool of likely characters.

Out of the six, temperature 0.5 and k=5 read as the most coherent, since keeping the distribution narrow keeps the model close to whatever it's most confident about. Temperature 1.5 produced the most varied output by far, with k=20 a close second. Both of those observations line up with what each method is actually doing to the probability distribution, rather than being a claim beyond what's printed above.

Final findings

The multi-head masked self-attention, decoder block, and full MiniGPT model that I built by hand meet every architectural requirement of this assignment: four heads with a head dimension of 32, four decoder blocks, learned token and positional embeddings, and a causal mask that I verified directly against the actual tensors rather than just assuming it worked. Training loss dropped every epoch, from 2.97 down to 1.83, but with only

1,609 overlapping windows from a single short excerpt and just six epochs of training, that wasn't enough to get fluent generated text out of the model. Every sample in Part 4 still looks clearly undertrained, and I've tried to say that plainly instead of overselling the results. The most consistent pattern across the decoding methods is that greedy decoding and the more restrictive sampling settings tend to repeat themselves, while higher temperature and larger k trade that repetition for more variety at the cost of coherence, which is exactly what each method is designed to do. No `nn.Transformer`, `nn.MultiheadAttention`, or HuggingFace transformer classes were used anywhere in this implementation.